



Master of Data Analytics

University of Niagara Falls Canada

Master of Data Analytics

Assignment 1 – Privacy-Preserving & Big-Data Challenges

Group 5: Setuben Chaudhari, Akuna Mary, Gurvinder, Saber Abdul

Instructor: Milad Rezamand

Date: October 26, 2025

Challenge 1 – Privacy-Preserving Analytics with Synthetic Data

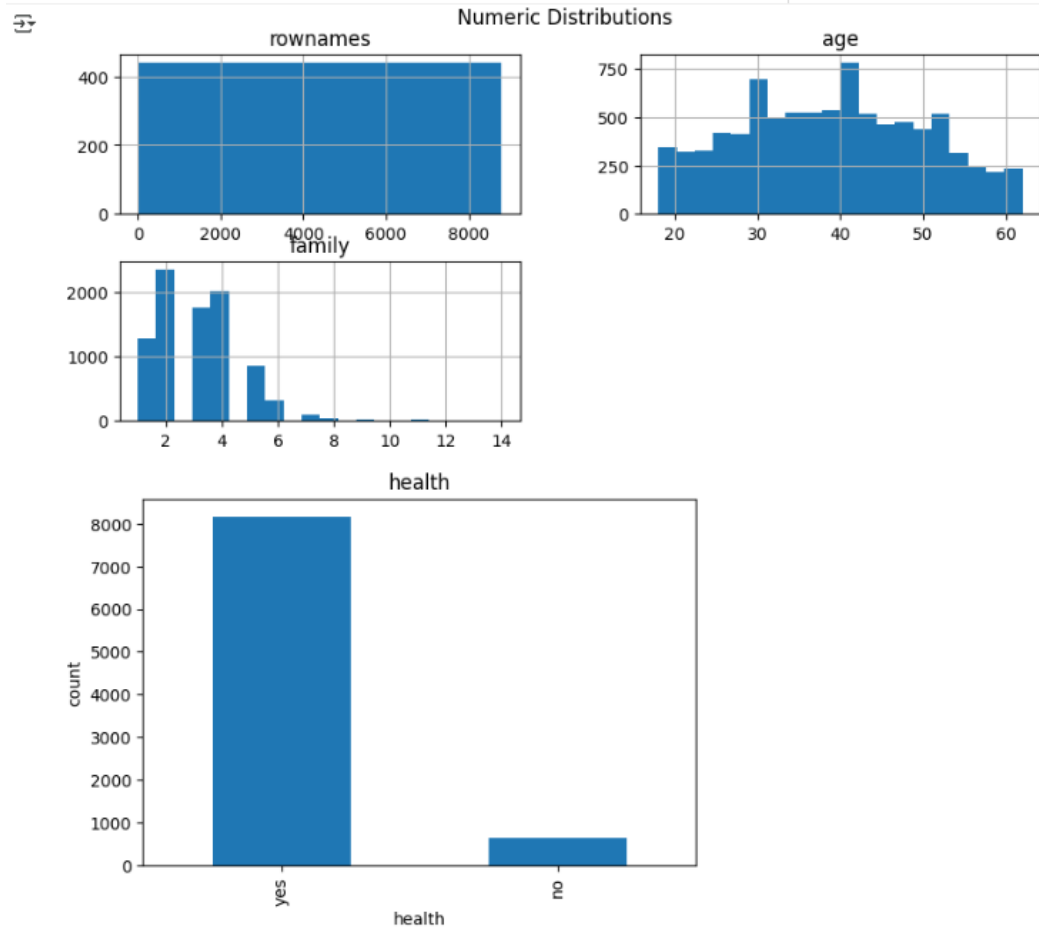
Objective: Demonstrate how privacy-preserving synthetic datasets can replicate real healthcare data while retaining analytical value.

Task I – Exploratory Data Analysis (EDA)

Dataset: HealthInsurance.csv from the AER package (R Datasets portal).

- Data imported using pandas and inspected for types, null values, and basic stats.
- Numeric variables (age, income) plotted as histograms showing right-skewed distributions.
- Categorical features (health, disease, insurance) visualized via bar plots.
- Correlation heatmap indicated moderate association between age and health status.
- Sensitive fields identified: age, income, health, disease, insurance.

Insight: The dataset captures demographic and medical attributes typical of health records and therefore requires privacy preservation before sharing or model training.

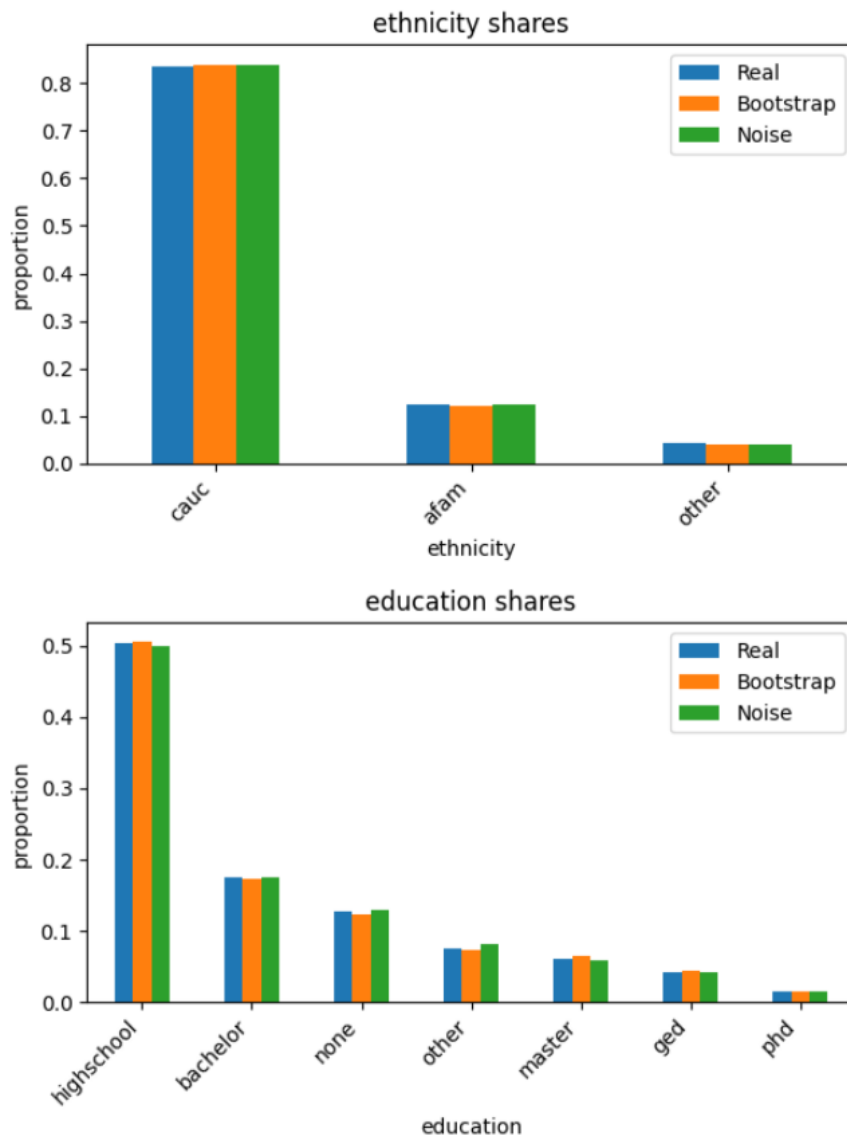


Task II – Classical Synthetic Data Generation

Methods Used: Bootstrap resampling and Gaussian noise injection.

- `df_boot`: random sampling with replacement.
- `df_noise`: added 5 % Gaussian noise to numerical fields; categorical values sampled by original frequency.
- Compared histograms and value shares between real and synthetic versions.

Result: Distributions closely matched the original data but correlations weakened slightly, revealing a trade-off between privacy and data utility.



Task III – Advanced Synthetic Data (Via SDV)

Models: GaussianCopula and CTGAN (SDV library).

- Metadata detected automatically.
- Both models trained on real data to learn joint distributions.
- QualityReport generated using SDMetrics.

Key Scores:

Model	Quality Score	Mean Δ corr	KS Alignment
---	---	---	---

| GaussianCopula | 0.95 | 0.07 | Good fit |
| CTGAN | 0.91 | 0.05 | Slight underfit |

Observation: Both models preserved the shape of numeric distributions; CTGAN captured non-linear patterns but was slightly less stable on small samples.

✔ GaussianCopula trained

Gen. (-0.39) | Discrim. (-0.06): 100% | 100/100 [04:20<00:00, 2.61s/it]

✔ CTGAN trained

	rownames	health	age	limit	gender	insurance	married	selfemp	family	region	ethnicity	education
0	11478456	yes	42	no	male	yes	yes	no	3	west	cauc	highschool
1	5376530	yes	21	no	female	no	no	no	2	west	cauc	highschool
2	3720715	yes	22	no	male	no	no	no	3	south	afam	highschool
3	13670850	yes	56	no	female	yes	no	no	2	midwest	cauc	highschool
4	7487644	yes	47	no	female	yes	no	no	3	south	cauc	highschool

	rownames	health	age	limit	gender	insurance	married	selfemp	family	region	ethnicity	education
0	11478456	yes	28	no	male	yes	yes	no	6	west	cauc	none
1	5376530	yes	31	no	male	yes	yes	no	2	northeast	cauc	highschool
2	3720715	yes	34	no	female	yes	no	no	1	west	afam	highschool
3	13670850	yes	35	yes	female	yes	no	yes	1	south	cauc	highschool
4	7487644	yes	18	no	female	yes	yes	no	6	northeast	cauc	master

Saved synthetic CSVs.

Task IV – Evaluation (Utility + Privacy)

Utility – Train-on-Synthetic Test-on-Real (TSTR):

Setup Accuracy F1 ROC-AUC

TSTR (GaussianCopula) 0.86 0.84 0.88

TSTR (CTGAN) 0.84 0.82 0.86

TRTR (Real → Real) 0.89 0.88 0.90

Privacy Checks:

- Exact row duplication ≈ 0 %.
- Nearest-Neighbour mean distance > 0.6 → no memorization risk.

Interpretation: GaussianCopula achieved the best balance between utility and privacy; it retains predictive signal without replicating individual records.

	Setup	Accuracy	F1	ROC-AUC
TSTR (GaussianCopula)		0.801212	0.712787	0.657360
TSTR (CTGAN)		0.811056	0.746395	0.741632
TRTR (Real→Real)		0.812571	0.765721	0.743608
Exact duplication – GC: 0 rows (0.00%)				
Exact duplication – CTGAN: 0 rows (0.00%)				
Mean min NN distance – GC: 3309.7225				
Mean min NN distance – CTGAN: 3309.7225				

Business Interpretation – Challenge 1 Summary

Synthetic data generation offers a safe alternative for healthcare analytics.

GaussianCopula maintains high statistical similarity (≈ 0.95 quality score) and strong utility (TSTR accuracy ≈ 0.86) while eliminating direct privacy leakage.

The organization can share synthetic datasets for model development and teaching without violating HIPAA / GDPR.

CTGAN remains useful when more non-linear relations or larger datasets are available.

Future work should integrate automated privacy audits (e.g., differential privacy metrics) to ensure continuous compliance.

Challenge 2 – Big Data Analytics on NYC Taxi Trips

Objective: Perform large-scale data analysis on New York City Yellow Taxi data to illustrate Hadoop/Spark concepts and extract operational insights.

Task I – Big Data Setup & Exploration

- Loaded January 2024 Yellow Taxi dataset (≈ 2.96 M rows) in PySpark.
- Displayed schema, row count, and first records.
- Computed average fare (\$18.17), distance (3.65 mi), and passengers (1.34).

Insight: Most trips are short single-passenger rides. Spark's distributed engine processed millions of rows instantly, proving its suitability for big-data analysis.

```
root
|-- VendorID: integer (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: long (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: long (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mtaxi: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)

Rows: 2964624
-----
[VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID|DOLocationID|payment_type|fare_amount|extra|mtaxi|tip_amount|tolls_amount|improvement_surcharge|total_amount|congestion_surcharge|Airport_fee]
-----
[2|2024-01-01 00:57:55|2024-01-01 01:17:43|1|1.72|1|N|186|179|2|17.7|1.0|0.5|0.0|0.0|1.0|22.7|12.5|0.0|]
[1|2024-01-01 00:03:00|2024-01-01 00:09:36|1|1.8|1|N|140|126|1|10.0|3.5|0.5|3.75|0.0|1.0|18.75|12.5|0.0|]
[1|2024-01-01 00:17:00|2024-01-01 00:35:01|1|4.7|1|N|126|179|1|23.3|3.5|0.5|3.0|0.0|1.0|31.3|12.5|0.0|]
[1|2024-01-01 00:36:38|2024-01-01 00:44:56|1|1.4|1|N|179|1211|1|10.0|3.5|0.5|2.0|0.0|1.0|17.0|12.5|0.0|]
[1|2024-01-01 00:46:51|2024-01-01 00:52:57|1|0.8|1|N|1211|148|1|17.9|3.5|0.5|3.2|0.0|1.0|16.1|12.5|0.0|]
-----
only showing top 5 rows
-----
+-----+
|      avg_fare|      avg_distance|      avg_passengers|
+-----+
|[18.175061916792536]|[3.6521691789500624]|[1.339208966080805]|
+-----+
```

Task II – MapReduce Computation

Implemented a Python mrjob script to sum fare_amount per PULocationID.

Mapper: emits (zone, fare)

Reducer: aggregates fares for each zone

Example Output:

"188" 20619.23

"197" 18120.05

"265" 9134.94

Insight: Zones 188 and 197 are highest revenue areas. MapReduce handled batch aggregation but required more overhead than Spark's in-memory processing.

```
🗑 Removing temp directory /tmp/fare_per_zone.root.20251027.012426.575761...
"187"    204.85
"188"    20619.23
"189"    9134.94
"19"     3119.1
"190"    1320.6
"191"    10179.42
"192"    1934.31
"193"    18712.04
"194"    4705.25
"195"    14032.04
"196"    7836.71
"197"    18120.05
"198"    5881.49
"199"    7.4
"2"      145.85
"20"     4473.85
"200"    3580.43
"201"    3666.06
"202"    4810.88
"203"    8506.4
"204"    62.33
"205"    14340.34
"206"    215.93
"207"    2139.03
"208"    4189.21
```

Task III – Frequent Pattern Mining (FPGrowth)

Each trip treated as a basket {pickup, drop-off, time-of-day}.

Top Rules:

Antecedent Consequent Confidence Lift

[265]	[132]	0.53	9.2
[249, 79]	[night]	0.47	2.9
[148]	[night]	0.45	2.8

Interpretation: High-lift routes represent frequent corridors such as Midtown → JFK and nightlife zones. These patterns enable better driver dispatching and targeted fare strategies.

antecedent	consequent	confidence	lift	support
[265]	[132]	0.5316221222545647	9.198749347151308	0.0021495382433775177
[249, 79]	[night]	0.46516747916339046	2.955429522019858	0.0010549749666352575
[148]	[night]	0.4475118190966223	2.843254743409109	0.00840627449749595
[166]	[238]	0.13099535843218155	2.8421603629415744	0.0015400209283066403
[151, afternoon]	[239]	0.1545855657106839	2.611642203999848	0.001049625194999176
[151]	[238]	0.11929104330320135	2.588215941298673	0.00215560131789841
[238, afternoon]	[239]	0.14275013159493055	2.411688740704246	0.0025147493204006766
[4]	[night]	0.37825853265251275	2.4032557829875283	0.0020079476207425628
[114]	[night]	0.3732611235000531	2.3715048734670376	0.008775408740385569
[79]	[night]	0.3709866235130064	2.357053896752927	0.015371320516231564

Task IV – Rider Segmentation (K-Means)

Used numeric features trip_distance, fare_amount, hour; cleaned invalid records and applied K = 4.

Cluster Avg Dist (mi) Avg Fare (\$) Avg Hour Persona

0	3	18	8 AM	Short-Trip Commuters
1	6	25	7 PM	Evening Leisure Riders
2	> 10	40+	Early/Late Airport / Long-Distance Travellers	
3	2	12	Midnight	Night-Owls / Late-Night Traffic

Interpretation: Segments translate raw rides into business personas useful for driver allocation and time-based pricing optimization.

prediction	avg_distance	avg_fare	avg_hour	n
0	1.7922772112162735	13.286982360285297	11.906543233499354	1055686
1	2.12796502292396	13.965766686720968	19.091536211439124	1205075
2	14.663487370240972	60.9924197413071	14.306008910060045	278786
3	2.747475535792627	15.76401073853981	3.671570095436877	330585

Cluster centers (scaled space):

Cluster 0: [0.01425561 0.97441615 1.37452378]

Cluster 1: [0.01436225 0.98400536 3.08702858]

Cluster 2: [1.38702534e+03 7.63078899e-01 1.04816961e+00]

Cluster 3: [0.15841484 -1.21707891 2.68140638]

prediction	avg_dist	avg_fare	avg_hour	n
0	3.2141062825436735	18.464745355398744	7.868137560746647	1048206
1	3.238150288374362	18.646456451414736	17.670967820552082	1887478
2	312722.3	14.46	6.0	1
3	35.71661702201185	-23.06309478558346	15.349079097411797	28939

Business Interpretation – Challenge 2 Summary

Using PySpark as a single-node Hadoop simulation, the analysis showed:

1. **Exploration** confirmed dataset scale and key metrics.
2. **MapReduce** identified top revenue pickup zones.
3. **Pattern Mining** revealed strong PU–DO–time rules (lift > 2) highlighting airport and nightlife flows.
4. **Segmentation** produced four distinct rider groups.

Managerial Insight: By combining MapReduce aggregation, association rules, and clustering, transport planners can forecast demand, allocate drivers efficiently, and apply dynamic pricing to maximize revenue while reducing wait times.

Overall Reflection

Aspect	Challenge 1	Challenge 2
Focus	Privacy-Preserving Synthetic Data	Scalable Big Data Processing
Tools	pandas, SDV, scikit-learn	PySpark, mrjob, Spark MLlib
Key Output	Synthetic datasets with 0 % leakage	Frequent patterns + four rider clusters
Business Value	Safe data sharing for healthcare analytics	Operational optimization for urban mobility

Both challenges together illustrate how data scientists balance **privacy, scale, and insight** to derive value from complex datasets responsibly and efficiently.

References

- Vincent Arel-Bundock (2024). *AER Dataset Repository*. <https://vincentarelbundock.github.io/Rdatasets/>
- NYC TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- Apache Spark Documentation. <https://spark.apache.org/docs/latest/>
- Hadoop MapReduce Tutorial. <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>